

data-wrangling

October 25, 2024

#DATA COLLECTION trough API*

```
[5]: import requests

def make_request(endpoint, payload=None):

    return requests.get(
        f'https://www.ncei.noaa.gov/cdo-web/api/v2/{endpoint}',
        headers={
            'token': 'AxQoejpxMSZkhVamvmsFEedwezjriTKtU'
        },
        params=payload
    )
```

```
[6]: response = make_request('datasets', {'startdate': '2018-10-01'})
response.status_code
```

```
[6]: 200
```

```
[7]: response.json().keys()
```

```
[7]: dict_keys(['metadata', 'results'])
```

```
[8]: response.json()['metadata']
```

```
[8]: {'resultset': {'offset': 1, 'count': 11, 'limit': 25}}
```

```
[9]: response.json()['results'][0].keys()
```

```
[9]: dict_keys(['uid', 'mindate', 'maxdate', 'name', 'datacoverage', 'id'])
```

```
[10]: [(data['id'], data['name']) for data in response.json()['results']]
```

```
[10]: [('GHCND', 'Daily Summaries'),
      ('GSOM', 'Global Summary of the Month'),
      ('GSOY', 'Global Summary of the Year'),
      ('NEXRAD2', 'Weather Radar (Level II)'),
      ('NEXRAD3', 'Weather Radar (Level III)'),
```

```
( 'NORMAL_ANN', 'Normals Annual/Seasonal'),
( 'NORMAL_DLY', 'Normals Daily'),
( 'NORMAL_HLY', 'Normals Hourly'),
( 'NORMAL_MLY', 'Normals Monthly'),
( 'PRECIP_15', 'Precipitation 15 Minute'),
( 'PRECIP_HLY', 'Precipitation Hourly')]
```

```
[11]: response = make_request(
        'datacategories',
        payload={
            'datasetid' : 'GHCND'
        }
    )
    response.status_code
```

[11]: 200

```
[12]: response.json()['results']
```

```
[12]: [{ 'name': 'Evaporation', 'id': 'EVAP'},
        { 'name': 'Land', 'id': 'LAND'},
        { 'name': 'Precipitation', 'id': 'PRCP'},
        { 'name': 'Sky cover & clouds', 'id': 'SKY'},
        { 'name': 'Sunshine', 'id': 'SUN'},
        { 'name': 'Air Temperature', 'id': 'TEMP'},
        { 'name': 'Water', 'id': 'WATER'},
        { 'name': 'Wind', 'id': 'WIND'},
        { 'name': 'Weather Type', 'id': 'WXTYPE'}]
```

```
[13]: response = make_request(
        'datatypes',
        payload={
            'datacategoryid' : 'TEMP',
            'limit' : 100
        }
    )
    response.status_code
```

[13]: 200

```
[14]: [(datatype['id'], datatype['name']) for datatype in response.
        ↪json()['results']][-5:]
```

```
[14]: [( 'MNTM', 'Monthly mean temperature'),
        ( 'TAVG', 'Average Temperature.'),
        ( 'TMAX', 'Maximum temperature'),
        ( 'TMIN', 'Minimum temperature'),
```

```
('TOBS', 'Temperature at the time of observation')]
```

```
[16]: response = make_request(  
      'locationcategories',  
      {  
        'datasetid' : 'GHCND'  
      }  
    )  
response.status_code
```

```
[16]: 200
```

```
[17]: import pprint  
pprint.pprint(response.json())
```

```
{'metadata': {'resultset': {'count': 12, 'limit': 25, 'offset': 1}},  
 'results': [{'id': 'CITY', 'name': 'City'},  
              {'id': 'CLIM_DIV', 'name': 'Climate Division'},  
              {'id': 'CLIM_REG', 'name': 'Climate Region'},  
              {'id': 'CNTRY', 'name': 'Country'},  
              {'id': 'CNTY', 'name': 'County'},  
              {'id': 'HYD_ACC', 'name': 'Hydrologic Accounting Unit'},  
              {'id': 'HYD_CAT', 'name': 'Hydrologic Cataloging Unit'},  
              {'id': 'HYD_REG', 'name': 'Hydrologic Region'},  
              {'id': 'HYD_SUB', 'name': 'Hydrologic Subregion'},  
              {'id': 'ST', 'name': 'State'},  
              {'id': 'US_TERR', 'name': 'US Territory'},  
              {'id': 'ZIP', 'name': 'Zip Code'}]}
```

```
[18]: def get_item(name, what, endpoint, start=1, end=None):  
      # find the midpoint which we use to cut the data in half each time  
      mid = (start + (end if end else 1)) // 2  
  
      # lowercase the name so this is not case-sensitive  
      name = name.lower()  
  
      payload = {  
        'datasetid' : 'GSOM',  
        'sortfield' : 'name',  
        'offset' : mid, # we will change the offset each time  
        'limit' : 1 # we only want one value back  
      }  
      # make our request adding any additional filter parameters from `what`  
      response = make_request(endpoint, {**payload, **what})  
  
      if response.ok:
```

```

    # if response is ok, grab the end index from the response metadata the
    ↪first time through
    end = end if end else response.json()['metadata']['resultset']['count']

    # grab the lowercase version of the current name
    current_name = response.json()['results'][0]['name'].lower()

    # if what we are searching for is in the current name, we have found our
    ↪item
    if name in current_name:
        return response.json()['results'][0] # return the found item

    else:
        if start >= end:
            # if our start index is greater than or equal to our end, we couldn't
            ↪find it
            return {}
        elif name < current_name:
            # our name comes before the current name in the alphabet, so we
            ↪search further to the left
            return get_item(name, what, endpoint, start, mid - 1)
        elif name > current_name:
            # our name comes after the current name in the alphabet, so we search
            ↪further to the right
            return get_item(name, what, endpoint, mid + 1, end)
        else:
            # response wasn't ok, use code to determine why
            print(f'Response not OK, status: {response.status_code}')

def get_location(name):
    return get_item(name, {'locationcategoryid' : 'CITY'}, 'locations')

```

```

[19]: nyc = get_location('New York')
      nyc

```

```

[19]: {'mindate': '1869-01-01',
      'maxdate': '2024-09-01',
      'name': 'New York, NY US',
      'datacoverage': 1,
      'id': 'CITY:US360019'}

```

```

[20]: central_park = get_item('NY City Central Park', {'locationid' : nyc['id']},
    ↪ 'stations')
      central_park

```

```
[20]: {'elevation': 42.7,
      'mindate': '1869-01-01',
      'maxdate': '2024-09-01',
      'latitude': 40.77898,
      'name': 'NY CITY CENTRAL PARK, NY US',
      'datacoverage': 1,
      'id': 'GHCND:USW00094728',
      'elevationUnit': 'METERS',
      'longitude': -73.96925}
```

```
[21]: response = make_request(
      'data',
      {
        'datasetid' : 'GSOM',
        'stationid' : central_park['id'],
        'locationid' : nyc['id'],
        'startdate' : '2018-10-01',
        'enddate' : '2018-10-31',
        'datatypeid' : ['TMIN', 'TMAX', 'TOBS'], # temperature at time of
        ↪ observation, min, and max
        'units' : 'metric',
        'limit' : 1000
      }
    )
response.status_code
```

```
[21]: 200
```

```
[22]: import pandas as pd

df = pd.DataFrame(response.json()['results'])
df.head()
```

```
[22]:
```

	date	datatype	station	attributes	value
0	2018-10-01T00:00:00	TMAX	GHCND:USW00094728	,,,W	17.5
1	2018-10-01T00:00:00	TMIN	GHCND:USW00094728	,,,W	11.0

```
[23]: df.datatype.unique()
```

```
[23]: array(['TMAX', 'TMIN'], dtype=object)
```

```
[24]: if get_item(
      'NY City Central Park', {'locationid' : nyc['id'], 'datatypeid': 'TOBS'},
      ↪ 'stations'
    ):
    print('Found!')
```

Found!

```
[25]: laguardia = get_item(
      'LaGuardia', {'locationid' : nyc['id']}, 'stations'
    )
laguardia
```

```
[25]: {'elevation': 3,
      'mindate': '1939-11-01',
      'maxdate': '2024-09-01',
      'latitude': 40.77945,
      'name': 'LAGUARDIA AIRPORT, NY US',
      'datacoverage': 1,
      'id': 'GHCND:USW00014732',
      'elevationUnit': 'METERS',
      'longitude': -73.88027}
```

```
[26]: response = make_request(
      'data',
      {
        'datasetid' : 'GHCND',
        'stationid' : laguardia['id'],
        'locationid' : nyc['id'],
        'startdate' : '2018-10-01',
        'enddate' : '2018-10-31',
        'datatypeid' : ['TMIN', 'TMAX', 'TAVG'], # temperature at time of
        ↪ observation, min, and max
        'units' : 'metric',
        'limit' : 1000
      }
    )
response.status_code
```

```
[26]: 200
```

```
[27]: df = pd.DataFrame(response.json()['results'])
df.head()
```

```
[27]:
```

	date	datatype	station	attributes	value
0	2018-10-01T00:00:00	TAVG	GHCND:USW00014732	H,,S,	21.2
1	2018-10-01T00:00:00	TMAX	GHCND:USW00014732	,,W,2400	25.6
2	2018-10-01T00:00:00	TMIN	GHCND:USW00014732	,,W,2400	18.3
3	2018-10-02T00:00:00	TAVG	GHCND:USW00014732	H,,S,	22.7
4	2018-10-02T00:00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1

```
[28]: df.datatype.unique()
```

```
[28]: array(['TAVG', 'TMAX', 'TMIN'], dtype=object)
```

```
[29]: df.datatype.value_counts()
```

```
[29]: datatype
TAVG    31
TMAX    31
TMIN    31
Name: count, dtype: int64
```

```
[30]: df.to_csv('nyc_temperatures.csv', index=False)
```

```
[32]: from google.colab import files
files.download('nyc_temperatures.csv')
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

#7.3 DATA CLEANING

```
[162]: import pandas as pd

df = pd.read_csv('nyc_temperatures.csv')
df.head()
```

```
[162]:
```

	date	datatype	station	attributes	value
0	2018-10-01T00:00:00	TAVG	GHCND:USW00014732	H,,S,	21.2
1	2018-10-01T00:00:00	TMAX	GHCND:USW00014732	,,W,2400	25.6
2	2018-10-01T00:00:00	TMIN	GHCND:USW00014732	,,W,2400	18.3
3	2018-10-02T00:00:00	TAVG	GHCND:USW00014732	H,,S,	22.7
4	2018-10-02T00:00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1

```
[163]: df.columns
```

```
[163]: Index(['date', 'datatype', 'station', 'attributes', 'value'], dtype='object')
```

Renaming Columns into desired column name.

```
[164]: df.rename(
    columns={
        'value' : 'Celcius',
        'attributes' : 'flags'
    }, inplace=True
)
```

```
[165]: df.columns
```

```
[165]: Index(['date', 'datatype', 'station', 'flags', 'Celcius'], dtype='object')
```

Converting column name into UPPER CASE

```
[166]: df.rename(str.upper, axis='columns').columns
```

```
[166]: Index(['DATE', 'DATATYPE', 'STATION', 'FLAGS', 'CELCIUS'], dtype='object')
```

```
[167]: print(df.dtypes)
```

```
date          object
datatype      object
station        object
flags          object
Celcius       float64
dtype: object
```

Converting Datatypes., `df.loc[]` - used for label-based indexing, `:` - means all rows, `'date'` - means the date column.

```
[168]: df['date'] = pd.to_datetime(df['date'], errors='coerce')
print(df.dtypes)
```

```
date          datetime64[ns]
datatype      object
station        object
flags          object
Celcius       float64
dtype: object
```

```
[169]: print(df.date.describe())
```

```
count          93
mean    2018-10-16 00:00:00
min      2018-10-01 00:00:00
25%      2018-10-08 00:00:00
50%      2018-10-16 00:00:00
75%      2018-10-24 00:00:00
max       2018-10-31 00:00:00
Name: date, dtype: object
```

```
[170]: pd.date_range(start='2018-10-01', periods=2, freq='D').tz_localize('UTC')
```

```
[170]: DatetimeIndex(['2018-10-01 00:00:00+00:00', '2018-10-02 00:00:00+00:00'],
dtype='datetime64[ns, UTC]', freq='D')
```

```
[171]: df['date'] = df['date'].dt.tz_localize('UTC') # First localize the date column
        ↳ to UTC
df['date'] = df['date'].dt.tz_convert('UTC') # Now convert to your desired
        ↳ timezone (if different from UTC)
print(df.head(10))
```


	date	datatype	station	flags	Celcius
0	2018-10-01 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.2
1	2018-10-01 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.6
2	2018-10-01 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.3
3	2018-10-02 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	22.7
4	2018-10-02 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1
5	2018-10-02 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	19.4
6	2018-10-03 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.8
7	2018-10-03 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.0
8	2018-10-03 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.9
9	2018-10-04 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.3

```
[172]: df.index = range(1, len(df) + 1)
print(df.head(10))
```

	date	datatype	station	flags	Celcius
1	2018-10-01 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.2
2	2018-10-01 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.6
3	2018-10-01 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.3
4	2018-10-02 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	22.7
5	2018-10-02 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1
6	2018-10-02 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	19.4
7	2018-10-03 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.8
8	2018-10-03 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.0
9	2018-10-03 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.9
10	2018-10-04 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.3

```
[173]: df['week_number'] = df['date'].dt.isocalendar().week
print(df.tail(10))
```

	date	datatype	station	flags	Celcius	\
84	2018-10-28 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	8.3	
85	2018-10-29 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	11.8	
86	2018-10-29 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	14.4	
87	2018-10-29 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	9.4	
88	2018-10-30 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	10.2	
89	2018-10-30 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	13.9	
90	2018-10-30 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	7.2	
91	2018-10-31 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	12.6	
92	2018-10-31 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	17.8	
93	2018-10-31 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	7.2	

	week_number
84	43
85	44
86	44
87	44
88	44

```

89          44
90          44
91          44
92          44
93          44

```

Adding new column, temp_F = Fahrenheit Temp. Calling it as new dataframe

```

[174]: df = df.assign(
        Farenhiet=(df.Celcius * 9/5) + 32
      )
      print(df.dtypes)

```

```

date          datetime64[ns, UTC]
datatype      object
station       object
flags         object
Celcius       float64
week_number   UInt32
Farenhiet     float64
dtype: object

```

```

[175]: df = df.assign(
        Celcius_whole=df.Celcius.astype('int'),
        Farenhiet_whole=lambda x: x.Farenhiet.astype('int')
      )
      print(df.head(10))

```

	date	datatype	station	flags	Celcius	\
1	2018-10-01 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.2	
2	2018-10-01 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.6	
3	2018-10-01 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.3	
4	2018-10-02 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	22.7	
5	2018-10-02 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1	
6	2018-10-02 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	19.4	
7	2018-10-03 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.8	
8	2018-10-03 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	25.0	
9	2018-10-03 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	18.9	
10	2018-10-04 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.3	

	week_number	Farenhiet	Celcius_whole	Farenhiet_whole
1	40	70.16	21	70
2	40	78.08	25	78
3	40	64.94	18	64
4	40	72.86	22	72
5	40	78.98	26	78
6	40	66.92	19	66
7	40	71.24	21	71
8	40	77.00	25	77

9	40	66.02	18	66
10	40	70.34	21	70

Converting datatype for datatype and station from object to Category.

```
[176]: df_with_categories = df.assign(
        station=df.station.astype('category'),
        datatype=df.datatype.astype('category')
    )
    print(df_with_categories.dtypes)
```

```
date                datetime64[ns, UTC]
datatype            category
station            category
flags              object
Celcius            float64
week_number        UInt32
Farenhiet          float64
Celcius_whole      int64
Farenhiet_whole    int64
dtype: object
```

Setting the category into order.

```
[177]: pd.Categorical(
        ['med', 'med', 'low', 'high'],
        categories=['low', 'med', 'high'],
        ordered=True
    )
```

```
[177]: ['med', 'med', 'low', 'high']
Categories (3, object): ['low' < 'med' < 'high']
```

```
[178]: df.sort_values(by='datatype', ascending=True).head(10)
```

```
[178]:
```

	date	datatype	station	flags	Celcius	\
1	2018-10-01 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.2	
22	2018-10-08 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	20.9	
25	2018-10-09 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	21.8	
67	2018-10-23 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	12.6	
28	2018-10-10 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	23.8	
64	2018-10-22 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	8.3	
31	2018-10-11 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	23.4	
61	2018-10-21 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	10.7	
34	2018-10-12 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	18.3	
58	2018-10-20 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	15.0	

	week_number	Farenhiet	Celcius_whole	Farenhiet_whole
1	40	70.16	21	70

22	41	69.62	20	69
25	41	71.24	21	71
67	43	54.68	12	54
28	41	74.84	23	74
64	43	46.94	8	46
31	41	74.12	23	74
61	42	51.26	10	51
34	41	64.94	18	64
58	42	59.00	15	59

```
[179]: df.nlargest(n=5, columns='Celcius')
```

```
[179]:
```

	date	datatype	station	flags	Celcius	\
20	2018-10-07 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	27.8	
29	2018-10-10 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	27.8	
32	2018-10-11 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.7	
5	2018-10-02 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1	
11	2018-10-04 00:00:00+00:00	TMAX	GHCND:USW00014732	,,W,2400	26.1	

	week_number	Farenhiet	Celcius_whole	Farenhiet_whole
20	40	82.04	27	82
29	41	82.04	27	82
32	41	80.06	26	80
5	40	78.98	26	78
11	40	78.98	26	78

```
[180]: df.nsmallest(n=5, columns=['Farenhiet'])
```

```
[180]:
```

	date	datatype	station	flags	Celcius	\
66	2018-10-22 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	5.6	
78	2018-10-26 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	5.6	
63	2018-10-21 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	6.1	
75	2018-10-25 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	6.1	
54	2018-10-18 00:00:00+00:00	TMIN	GHCND:USW00014732	,,W,2400	6.7	

	week_number	Farenhiet	Celcius_whole	Farenhiet_whole
66	43	42.08	5	42
78	43	42.08	5	42
63	42	42.98	6	42
75	43	42.98	6	42
54	42	44.06	6	44

```
[181]: df.sample(5, random_state=0).index
```

```
[181]: Index([3, 31, 56, 17, 14], dtype='int64')
```

```
[182]: df.sample(5, random_state=0).sort_index().index
```

```
[182]: Index([3, 14, 17, 31, 56], dtype='int64')
```

```
[183]: df.sort_index(axis=1).head()
```

```
[183]:
```

	Celcius	Celcius_whole	Farenhiet	Farenhiet_whole	datatype	\
1	21.2	21	70.16	70	TAVG	
2	25.6	25	78.08	78	TMAX	
3	18.3	18	64.94	64	TMIN	
4	22.7	22	72.86	72	TAVG	
5	26.1	26	78.98	78	TMAX	

	date	flags	station	week_number
1	2018-10-01 00:00:00+00:00	H,,S,	GHCND:USW00014732	40
2	2018-10-01 00:00:00+00:00	,,W,2400	GHCND:USW00014732	40
3	2018-10-01 00:00:00+00:00	,,W,2400	GHCND:USW00014732	40
4	2018-10-02 00:00:00+00:00	H,,S,	GHCND:USW00014732	40
5	2018-10-02 00:00:00+00:00	,,W,2400	GHCND:USW00014732	40

`.reset_index()` making new index for the selection, but not dropping the old index

```
[184]: df = df.sort_values(by='datatype').reset_index()  
print(df.head(10))
```

	index	date	datatype	station	flags	\
0	1	2018-10-01 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
1	22	2018-10-08 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
2	25	2018-10-09 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
3	67	2018-10-23 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
4	28	2018-10-10 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
5	64	2018-10-22 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
6	31	2018-10-11 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
7	61	2018-10-21 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
8	34	2018-10-12 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	
9	58	2018-10-20 00:00:00+00:00	TAVG	GHCND:USW00014732	H,,S,	

	Celcius	week_number	Farenhiet	Celcius_whole	Farenhiet_whole
0	21.2	40	70.16	21	70
1	20.9	41	69.62	20	69
2	21.8	41	71.24	21	71
3	12.6	43	54.68	12	54
4	23.8	41	74.84	23	74
5	8.3	43	46.94	8	46
6	23.4	41	74.12	23	74
7	10.7	42	51.26	10	51
8	18.3	41	64.94	18	64
9	15.0	42	59.00	15	59

```
[186]: print(df.describe())
```

	index	Celcius	week_number	Farenhiet	Celcius_whole \
count	93.000000	93.000000	93.0	93.000000	93.000000
mean	47.000000	15.408602	41.741935	59.735484	14.956989
std	26.990739	6.133703	1.301364	11.040666	6.084396
min	1.000000	5.600000	40.0	42.080000	5.000000
25%	24.000000	10.200000	41.0	50.360000	10.000000
50%	47.000000	14.400000	42.0	57.920000	14.000000
75%	70.000000	21.100000	43.0	69.980000	21.000000
max	93.000000	27.800000	44.0	82.040000	27.000000

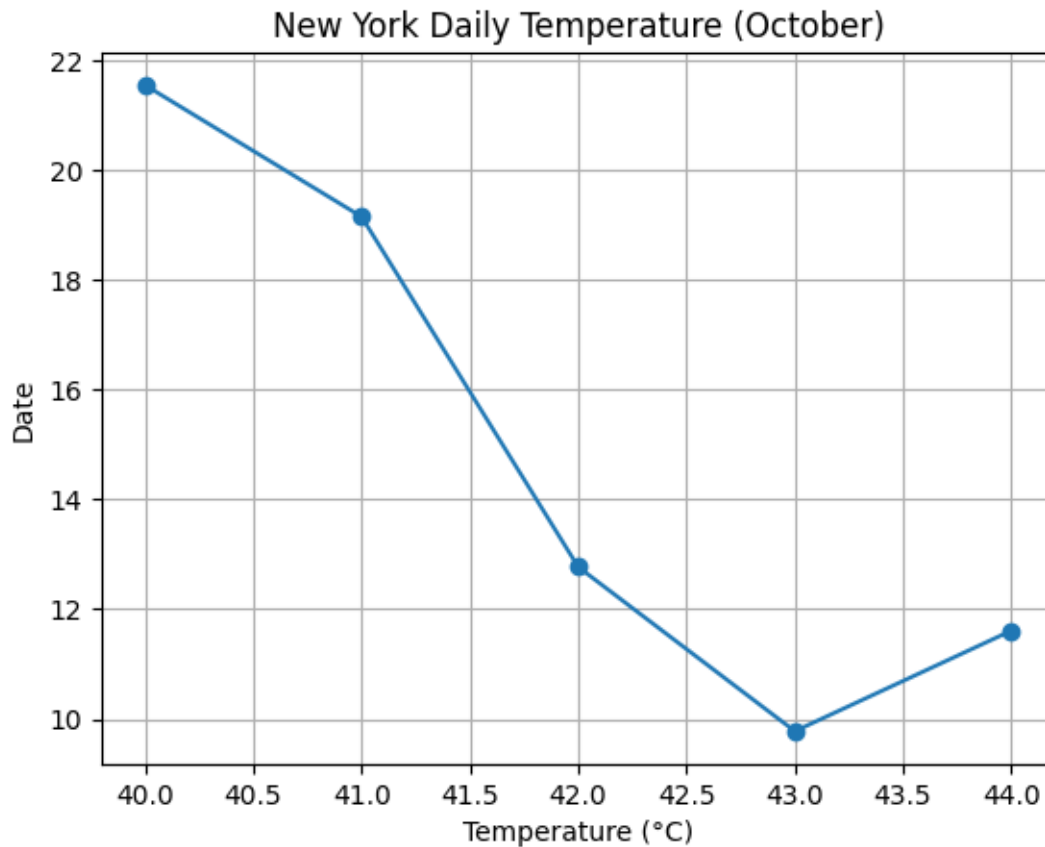
	Farenhiet_whole
count	93.000000
mean	59.215054
std	11.128547
min	42.000000
25%	50.000000
50%	57.000000
75%	69.000000
max	82.000000

```
[187]: import pandas as pd
import matplotlib.pyplot as plt
```

```
[191]: weekly_avg = df.groupby('week_number')['Celcius'].mean().reset_index()
print(weekly_avg)
```

	week_number	Celcius
0	40	21.557143
1	41	19.157143
2	42	12.771429
3	43	9.776190
4	44	11.611111

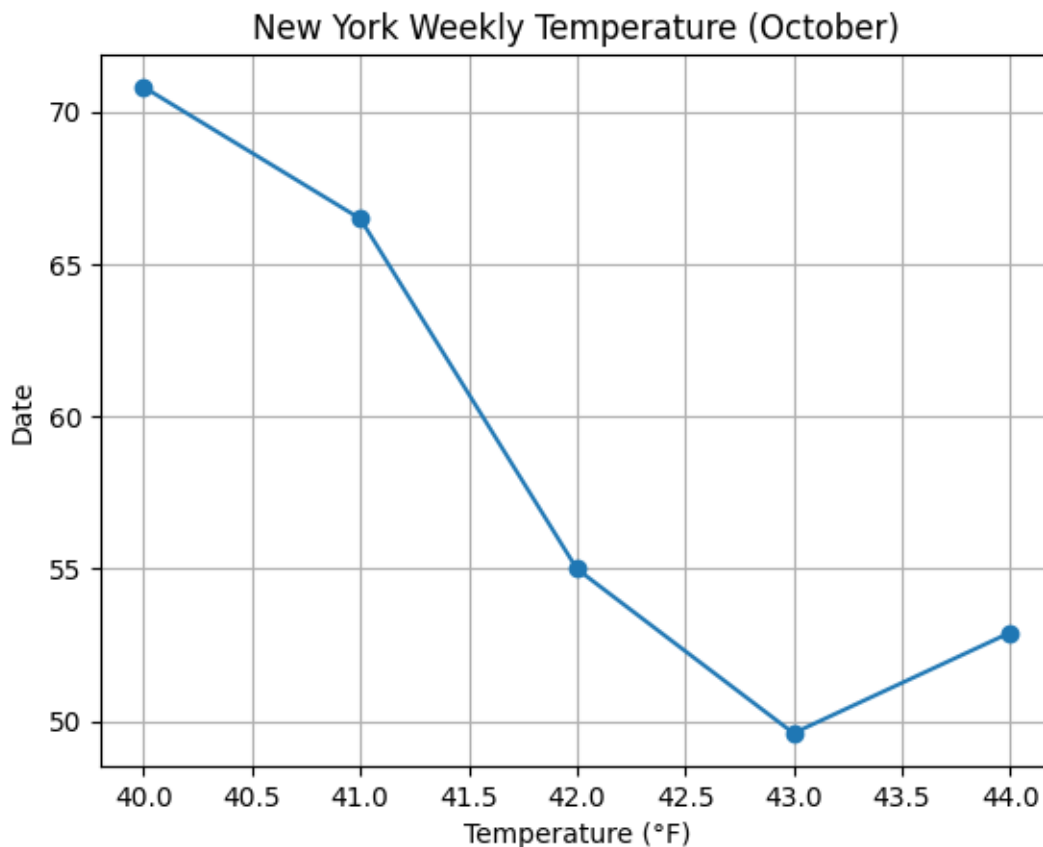
```
[193]: plt.plot(weekly_avg['week_number'], weekly_avg['Celcius'], marker='o')
plt.xlabel('Temperature (°C)')
plt.ylabel('Date')
plt.title('New York Weekly Temperature (October)')
plt.grid(True)
plt.show()
```



```
[195]: weekly_avg2 = df.groupby('week_number')['Farenhiet'].mean().reset_index()
print(weekly_avg2)
```

	week_number	Farenhiet
0	40	70.802857
1	41	66.482857
2	42	54.988571
3	43	49.597143
4	44	52.900000

```
[198]: plt.plot(weekly_avg2['week_number'], weekly_avg2['Farenhiet'], marker='o')
plt.xlabel('Temperature (°F)')
plt.ylabel('Date')
plt.title('New York Weekly Temperature (October)')
plt.grid(True)
plt.show()
```



```
[199]: df.describe()
```

```
[199]:
```

	index	Celcius	week_number	Farenhiet	Celcius_whole \
count	93.000000	93.000000	93.0	93.000000	93.000000
mean	47.000000	15.408602	41.741935	59.735484	14.956989
std	26.990739	6.133703	1.301364	11.040666	6.084396
min	1.000000	5.600000	40.0	42.080000	5.000000
25%	24.000000	10.200000	41.0	50.360000	10.000000
50%	47.000000	14.400000	42.0	57.920000	14.000000
75%	70.000000	21.100000	43.0	69.980000	21.000000
max	93.000000	27.800000	44.0	82.040000	27.000000

	Farenhiet_whole
count	93.000000
mean	59.215054
std	11.128547
min	42.000000
25%	50.000000
50%	57.000000
75%	69.000000

max 82.000000

```
[200]: df.to_csv('nyc_temperatures.csv', index=False)
```

```
[201]: from google.colab import files  
files.download('nyc_temperatures.csv')
```

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>